

TRADEBAY

Identity is the foundation of TradeBay. Every other domain (CRM, Inventory, procurement, finance) trusts Identity to answer: **who is this person?, which company are they acting for?, and what may they do?**

A **User** is a person who logs in. A **BusinessAccount** is a company that trades (buyer, supplier, or platform). They connect only through **Membership** — one person can belong to several companies, with a different **Role** in each. Permissions are atomic resource.action codes on roles — never if role == "admin".

Core rule: Authorization always checks permission codes on the active membership. Platform-only codes (e.g. suppliers.verify, settings.manage) are never granted to trading-company roles.

Key Mongo collections

users — email, password_hash, profile, status, email_verified_at
business_accounts — company type (buyer/supplier/platform), status, contact, address
business_memberships — (user_id, business_account_id, role_id, status)
roles + role_permissions + permissions — RBAC catalog per company
invitations — hashed token, role, expiry; invitee may use a different delivery email
sessions — refresh_token_hash, active_business_account_id, device metadata
auth_tokens — one-time OTP challenges (email_verification / password_reset)
audit_logs — who did what, on which business, from which IP
supplier_profiles — verification status + submitted documents

Read the code in this order

models.py / constants.py → permissions.py → dependencies.py → service.py → directory.py → auth_router.py / business_router.py / identity_router.py

Identity Domain

Learning Guide — APIs, Scenarios & Implementation

Every protected endpoint goes through FastAPI dependencies in dependencies.py. Understanding this path is the fastest way to become confident reading Identity.

1. **Extract access token** — Authorization: Bearer ... or HttpOnly access cookie.
2. **Decode JWT** — validates signature/expiry; payload carries user_id + session_id.
3. **Load session** — must exist, not revoked, not expired (cached briefly in auth_cache.py).
4. **Load user** — must be active (not suspended/deactivated); email may need verification for some flows.
5. **Resolve active business** — from session.active_business_account_id; load membership + role + permission codes into AuthContext.
6. **require_permission(resource, action)** — raises if the code is missing; also requires an operational business context.
7. **Handler** — thin; calls AuthService / BusinessService / DirectoryService.

AuthContext fields you will see everywhere: user, session, business, membership, role, permissions: set[str], plus helpers user_id, business_id, has_permission().

Cookies: Login/register/refresh set HttpOnly access + refresh cookies. Logout clears them. Refresh rotates the refresh token (hash stored on the session). SPA uses credentials: 'include'.

Backend focus · Front-end insight · Generated 2026-09-21

3. Backend architecture (layers)

Layer	Files	Responsibility
HTTP	auth_router, business_router, identity_router	Validate body, call service, wrap success(), set cookies
Auth deps	dependencies.py, guards.py	AuthContext, require_permission, require_seller, optional user
Services	service.py (Auth + Business), directory.py	Business rules, invitations, RBAC mutations, sessions, verification
Data	repository.py, models.py	Mongo CRUD; document shapes; indexes via db/indexes
Catalog	permissions.py, constants.py	Permission codes, system roles, statuses, OTP TTLs
Support	email.py, password.py, rate_limit.py, auth_cache.py, company_domain.py	Mail templates, hashing, challenge rate limits, domain rules

Routers are mounted in `app/api/router.py` under `API_V1_PREFIX (/api/v1)`.

4. API catalog — Authentication (/auth)

File: `auth_router.py` → `AuthService`. Legacy path aliases exist (`/verify-email`, etc.) with `include_in_schema=False`.

Method / Path	Auth	What it does
POST /auth/register	Public	Create user (+ optional company or accept invitation). Sets auth cookies. May auto-create buyer/supplier and seed Business Admin membership.
POST /auth/login	Public	Email + password → session + cookies. Rejects inactive accounts.
POST /auth/logout	Session	Revoke current session; clear cookies.
POST /auth/logout-all	Session	Revoke all sessions for the user; clear cookies.
POST /auth/refresh	Refresh cookie	Rotate refresh token; issue new access token + cookies.
GET /auth/me	Session	Canonical session context: user, active business, role, permissions. Prefer over GET /me.
POST /auth/email/verify	Public (OTP)	Consume one-time email verification token/OTP.
POST /auth/email/resend	Public / rate-limited	Resend verification challenge (opaque success to avoid email enumeration).
POST /auth/forgot-password	Public / rate-limited	Issue password-reset challenge if email exists (always generic response).
POST /auth/password/reset	Public (OTP)	Set new password with reset token; invalidates other challenges/sessions as designed.
POST /auth/password/change	Session	Authenticated password change (current + new).

5. API catalog — Businesses (/businesses)

File: `business_router.py` → `BusinessService` + switch via `AuthService.switch_business`.

Method / Path	Auth	What it does
GET /businesses	Session	List companies the user belongs to (memberships).
POST /businesses	Session	Create a company for the current user; seeds system roles; creator becomes Business Admin.
GET /businesses/current	Session	Return the session's active business (or null).
POST /businesses/switch	Session	Set <code>session.active_business_account_id</code> to a membership the user owns. Next requests use that RBAC.
GET /businesses/{id}	Membership scope	Fetch a company the user belongs to.
PATCH /businesses/{id}	businesses.manage	Update identity fields (name, contacts, address, tax ids, logo, etc.).
POST .../verification/submit	Supplier + manage	Submit supplier docs for platform review (status → pending).

Method / Path	Auth	What it does
DELETE .../verification/documents/{type}	Supplier + manage	Withdraw one doc while review is still pending.

6. API catalog — Members, Roles, Invitations, Sessions

All company-scoped routes use the **active business** from the session.

Members — /members

Method / Path	Auth	What it does
GET /members	users.read	List memberships (filter/paginate).
GET /members/{id}	users.read	View one membership.
PATCH /members/{id}	users.update	Change member role (cannot orphan last admin).
DELETE /members/{id}	users.remove	Remove membership.
POST /members/{id}/suspend	users.update	Suspend membership in this company.
POST /members/{id}/reactivate	users.update	Reactivate suspended membership.

Roles & permissions — /roles, /permissions

Method / Path	Auth	What it does
GET /roles	roles.read	List roles for active company (system + custom).
GET /roles/{id}	roles.read	Role detail + permission codes.
POST /roles	roles.manage	Create custom role; grants must be subset of actor's permissions.
PATCH /roles/{id}	roles.manage	Update grants (system roles editable carefully).
DELETE /roles/{id}	roles.manage	Delete custom role only (not system).
GET /permissions	roles.read	Global permission catalog.

Invitations — /invitations

Method / Path	Auth	What it does
GET /invitations	users.read	List invites for active company.
POST /invitations	users.invite	Invite by email + role; emails hashed token link.
GET /invitations/preview?token=	Public	Preview invite before accept (company/role).
POST /invitations/accept	Session (invitee)	Accept via body.token from email.
POST /invitations/decline	Session / token	Decline via token.
POST /invitations/{id}/resend	users.invite	Resend pending invite.
POST /invitations/{id}/revoke	users.invite	Revoke pending invite.
GET /invitations/{id}	Session	View one invite (scoped).

Account & sessions — /me, /sessions, /audit-logs

Method / Path	Auth	What it does
PATCH /me	Session	Update personal profile (name, phone, avatar).
GET /sessions	Session	List active sessions (mark current).
DELETE /sessions/{id}	Session	Revoke one session.
DELETE /sessions	Session	Revoke other sessions; keep current.
GET /audit-logs	audit_logs.read	Company audit trail for active business.
GET /audit-logs/{id}	audit_logs.read	One audit event.

7. API catalog — Platform staff (/platform)

Platform operators act inside the TradeBay platform tenant. Endpoints enforce platform business type + platform permissions (e.g. `suppliers.verify`, `businesses.read`, `users.update`).

Method / Path	Auth	What it does
GET /platform/me	Platform session	Platform membership context.

Method / Path	Auth	What it does
GET/POST /platform/businesses	Staff	List / provision buyer or supplier + owner admin.
GET /platform/businesses/{id}	Staff	Trading company profile.
GET .../members, .../roles	Staff	Inspect/manage another company's directory.
POST/PATCH/DELETE .../roles	Staff	Create/update/delete roles on a trading company.
PATCH .../members/{id}/role	Staff	Change a member's role remotely.
GET .../audit-logs, POs, RFQs	Staff	Oversight of company activity / commerce.
GET/POST /platform/users	Staff	List / provision user accounts.
POST /platform/users/suspend	Staff	Suspend any user.
POST /platform/users/{id}/reactivate	Staff	Reactivate suspended user.
GET /platform/suppliers	suppliers.verify	Verification queue.
POST /platform/suppliers/{id}/review	suppliers.verify	Approve or reject supplier verification.
GET /platform/audit-logs	Staff	Platform-tenant audit (not company logs).

8. End-to-end scenarios (how pieces connect)

Scenario A — Register a new buyer company

- Client POST /auth/register with email, password, names, business_name, business_type=buyer.
- AuthService creates user (pending/active per rules), hashes password, may create business_account.
- Seeds system roles for that business from permissions.py grants; creates Business Admin membership.
- Issues session + JWTs; sets cookies. Audit: user_registered / business_created.
- Email verification challenge may be issued (auth_tokens). Client later POST /auth/email/verify.
- Subsequent APIs use GET /auth/me to hydrate UI permissions.

Scenario B — Invite a teammate

- Admin switches to company (POST /businesses/switch) so session has the right active business.
- POST /invitations with invited_email, role_id (optional delivery_email).
- DirectoryService stores token_hash, status=pending, expires_at (+7 days). Emails link with raw token.
- Invitee opens link → GET /invitations/preview?token=... (public).
- If new: register with invitation_token. If existing: login then POST /invitations/accept {token}.
- Accept creates/activates membership with invited role; invitation → accepted. Cannot escalate beyond inviter's grants.

Scenario C — Switch company context

- User belongs to Company A (buyer) and Company B (supplier).
- POST /businesses/switch { business_id: B } updates session.active_business_account_id.
- Next GET /auth/me returns B's role + permissions. Catalog/procurement now authorize as supplier.
- No client-side "fake switch" — server session is source of truth (frontend AuthProvider documents this).

Scenario D — Custom role with least privilege

- Actor has roles.manage. POST /roles with name + permission code list.
- DirectoryService.assert_subset ensures requested codes $\subseteq$ actor.permissions (and not platform-only).
- role_permissions rows link role → permission ids. Members assigned via PATCH /members/{id}.
- Deleting a role fails if system role or still assigned (service guards).

Scenario E — Supplier verification

- Supplier admin POST /businesses/{id}/verification/submit with document metadata/payload.
- supplier_profiles.verification_status → pending (see VERIFICATION_TRANSITIONS in constants).
- Platform staff GET /platform/suppliers then POST /platform/suppliers/{id}/review {approve|reject}.
- Selling gates (require_seller) block unverified suppliers from seller-only catalog mutations.

Scenario F — Password reset

- POST /auth/forgot-password { email } — always generic OK; rate-limited.
- If user exists: auth_token purpose=password_reset (hashed), email OTP/link.
- POST /auth/password/reset { token, new_password } — consume token, update hash, typically revoke sessions.

Scenario G — Platform provisions a trading company

- Staff POST /platform/businesses with account_type, business_name, owner_* credentials.
- AuthService.provision_trading_account_by_platform creates user + company + admin membership in one flow.
- Useful for onboarding without public self-serve when ops creates accounts.

9. Implementation details that matter

9.1 Sessions & tokens

Access JWT is short-lived and carries session id. Refresh token is a random secret; only its hash lives on sessions. Refresh rotates the hash (reuse detection patterns depend on service logic). Logout sets revoked_at. Device IP/UA stored for session list UI.

9.2 Challenges (OTP)

Email verification and password reset use auth_tokens with purpose enum, attempt counters (max 5), and TTLs (~15 minutes for OTP). Failed attempts register; consumed tokens set used_at. Rate limits in rate_limit.py protect resend/forgot.

9.3 RBAC seeding

On company create, system roles are inserted (Business Admin, Sales Manager, Sales Rep, Finance, Viewer, etc.) with grants from `permissions.py`. Platform Admin / Operator exist on the platform tenant. Permission catalog is seeded at API startup from `DEFAULT_PERMISSION_CATALOG`.

9.4 Safety rails

- Cannot remove/suspend the last Business Admin (`assert_not_last_admin`).
- Cannot assign permissions you do not hold (`assert_subset`).
- Platform-only codes stripped from trading roles.
- Business must be operational (verified/pending/active) for trading APIs.
- `EmailUnverifiedError` / `AccountInactiveError` / `SellingNotVerifiedError` for clear client UX.
- Audit log on sensitive mutations (invite, role change, verification review, suspend).

9.5 Auth cache

`auth_cache.py` memoizes session auth snapshots and role→permission sets briefly to cut Mongo round-trips on hot paths. Invalidation follows session/role writes.

9.6 Company domain helpers

`company_domain.py` encodes business-type rules (e.g. what a supplier vs buyer may configure). Keep domain decisions here rather than scattering in routers.

10. Frontend insight (short)

You do not need the full UI tree to understand Identity — the frontend mirrors the backend session model.

- **AuthProvider** (`providers/AuthProvider.tsx`) — loads GET `/auth/me` via React Query; exposes user, business, permissions, `hasPermission()`, `logout`, `refreshSession`.
- **authApi / identityApi** — thin wrappers over `/auth`, `/businesses`, `/members`, `/roles`, `/invitations`, `/platform`.
- **DashboardGate** — redirects unauthenticated users to login unless path is in `guestAccess.ts` (`marketplace/categories/suppliers/planner`).
- **PermissionGate** — page-level check for `resource.action`; optional `allowGuest` for explore pages.
- **Company space UI** — `IdentitySidebar` + pages under `/members`, `/roles`, `/invitations`, `/businesses`, `/audit` call the directory APIs with the active company already selected on the session.
- **Cookies** — api client uses credentials include; no access token in `localStorage`.

Confidence tip: When a UI button is missing, check `hasPermission('...')` first — then confirm the session's active business is the company you think it is (`switcher` → `POST /businesses/switch`).

11. Where to look in the repo

Path	Why open it
<code>backend/app/modules/identity/</code>	Entire domain package (start at <code>__init__.py</code> docstring)
<code>backend/app/api/router.py</code>	How routers mount under <code>/api/v1</code>
<code>backend/app/core/security.py</code>	JWT encode/decode used by dependencies
<code>backend/tests/integration/test_identity.py</code>	Happy-path flows (register, invite, switch)
<code>backend/tests/integration/test_identity_security.py</code>	Authz edge cases, escalation guards
<code>frontend/lib/api/identityApi.ts</code> + <code>authApi.ts</code>	Client surface area matching these APIs
<code>frontend/providers/AuthProvider.tsx</code>	Session hydration for the SPA

12. Quick study checklist

- Trace one `POST /auth/login` from router → `AuthService.login` → `_issue_session` → cookies.
- Trace `require_permission` in a members route and see `AuthContext.permissions` populate.
- Create an invite in the UI (or via API) and accept it with a second user.
- Switch businesses and watch `/auth/me` permissions change.
- Read `PLATFORM_ONLY` in `permissions.py` and explain why trading roles cannot get `suppliers.verify`.
- Skim `test_identity_security.py` for the failure cases the design cares about.

Generated 2026-09-21 from the TradeBay codebase. Paths and summaries reflect the Identity routers as mounted under `/api/v1`.